

DEVOPS CAPSTONE PROJECT

Employee Management Portal

Final Project Flow

1. Phase 1 — Application Source Code
2. Phase 2 — Push Code to GitHub
3. Phase 3 — Create Dockerfiles
4. Phase 4 — Push Docker Images to Docker Hub
5. Phase 5 — Build CI/CD Pipelines
6. Phase 6 — Deploy Application on Kubernetes
7. Phase 7 — Provision AWS Infrastructure Using Terraform
8. Phase 8 — Use Jenkins and Ansible to Install Monitoring Tools
9. Phase 9 — Configure Prometheus, Grafana, Node Exporter, and cAdvisor

The AWS infrastructure created through Terraform will be configured using Ansible. These AWS machines will be used to monitor the application deployed on Kubernetes.

PHASE 1 — APPLICATION SOURCE CODE

Students will create a simple Employee Management Portal with:

- React frontend.
- Node.js and Express backend.
- Employee listing.
- Search functionality.
- Department filtering.
- REST API.
- Health-check endpoint.

Final Application Architecture

React Frontend

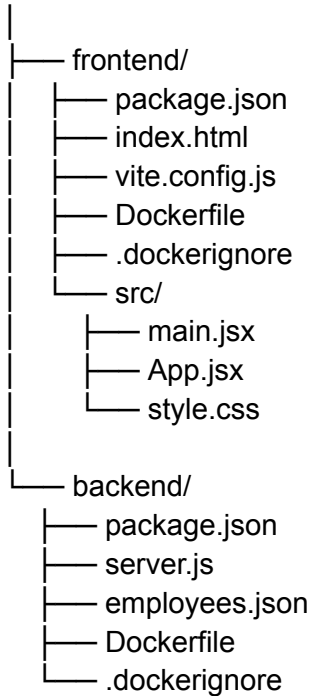


Node.js / Express Backend



1. Project Directory Structure

employee-management-devops/



2. Backend Source Code

File 1: `backend/package.json`

JSON

```
{
  "name": "employee-management-backend",
  "version": "1.0.0",
  "description": "Backend API for the Employee Management Portal",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "node --watch server.js"
  },
  "dependencies": {
```

```
"cors": "^2.8.5",  
"express": "^4.21.2"  
}  
}
```

File 2: backend/employees.json

JSON

```
[  
  {  
    "id": 1,  
    "name": "Ali Khan",  
    "email": "ali.khan@example.com",  
    "department": "Engineering",  
    "role": "DevOps Engineer",  
    "location": "Islamabad",  
    "experience": 3  
  },  
  {  
    "id": 2,  
    "name": "Sara Ahmed",  
    "email": "sara.ahmed@example.com",  
    "department": "Finance",  
    "role": "Financial Analyst",  
    "location": "Lahore",  
    "experience": 4  
  },  
  {  
    "id": 3,  
    "name": "Usman Raza",  
    "email": "usman.raza@example.com",  
    "department": "Engineering",  
    "role": "Backend Developer",  
    "location": "Karachi",  
    "experience": 2  
  },  
  {  
    "id": 4,  
    "name": "Ayesha Malik",  
    "email": "ayesha.malik@example.com",  
    "department": "Human Resources",  
    "role": "HR Manager",  
    "location": "Islamabad",  
  }  
]
```

```
    "experience": 6
  },
  {
    "id": 5,
    "name": "Hamza Tariq",
    "email": "hamza.tariq@example.com",
    "department": "Marketing",
    "role": "Marketing Specialist",
    "location": "Rawalpindi",
    "experience": 3
  },
  {
    "id": 6,
    "name": "Hina Shah",
    "email": "hina.shah@example.com",
    "department": "Engineering",
    "role": "Frontend Developer",
    "location": "Lahore",
    "experience": 2
  },
  {
    "id": 7,
    "name": "Bilal Ahmed",
    "email": "bilal.ahmed@example.com",
    "department": "Operations",
    "role": "Operations Manager",
    "location": "Karachi",
    "experience": 7
  },
  {
    "id": 8,
    "name": "Maham Iqbal",
    "email": "maham.iqbal@example.com",
    "department": "Finance",
    "role": "Accountant",
    "location": "Islamabad",
    "experience": 4
  }
]
```

File 3: backend/server.js

JavaScript

```
const express = require("express");
const cors = require("cors");
const employees = require("./employees.json");

const app = express();

const PORT = process.env.PORT || 5000;
const APP_NAME =
  process.env.APP_NAME || "Employee Management API";
const APP_ENV =
  process.env.APP_ENV || "development";

app.use(cors());
app.use(express.json());

app.get("/", (req, res) => {
  res.json({
    message: `${APP_NAME} is running`,
    environment: APP_ENV
  });
});

app.get("/health", (req, res) => {
  res.status(200).json({
    status: "healthy",
    service: "employee-backend",
    environment: APP_ENV
  });
});

app.get("/api/employees", (req, res) => {
  const { search, department } = req.query;

  let result = employees;

  if (search) {
    const searchValue = search.toLowerCase();

    result = result.filter((employee) => {
      return (
        employee.name.toLowerCase().includes(searchValue) ||
        employee.role.toLowerCase().includes(searchValue) ||
        employee.email.toLowerCase().includes(searchValue)
      );
    });
  }
});
```

```

    });
  }

  if (department && department !== "All") {
    result = result.filter(
      (employee) => employee.department === department
    );
  }

  res.status(200).json(result);
});

app.get("/api/employees/:id", (req, res) => {
  const employeeId = Number(req.params.id);

  const employee = employees.find(
    (item) => item.id === employeeId
  );

  if (!employee) {
    return res.status(404).json({
      message: "Employee not found"
    });
  }

  res.status(200).json(employee);
});

app.get("/api/departments", (req, res) => {
  const departments = [
    "All",
    ...new Set(employees.map((employee) => employee.department))
  ];

  res.status(200).json(departments);
});

app.use((req, res) => {
  res.status(404).json({
    message: "Route not found"
  });
});

app.listen(PORT, "0.0.0.0", () => {

```

```
console.log(`${APP_NAME} started`);
console.log(`Environment: ${APP_ENV}`);
console.log(`Listening on port: ${PORT}`);
});
```

File 4: backend/.dockerignore

```
node_modules
npm-debug.log
.git
.gitignore
.env
Dockerfile
```

3. Frontend Source Code

The frontend will use:

- React.
- Vite.
- JavaScript.
- CSS.
- Fetch API.

The frontend will communicate with the backend through an environment variable named:

VITE_API_URL

File 5: frontend/package.json

JSON

```
{
  "name": "employee-management-frontend",
  "version": "1.0.0",
  "description": "Frontend for the Employee Management Portal",
  "private": true,
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
```

```
"dependencies": {
  "react": "^18.3.1",
  "react-dom": "^18.3.1"
},
"devDependencies": {
  "@vitejs/plugin-react": "^4.3.4",
  "vite": "^6.0.7"
}
}
```

File 6: frontend/index.html

HTML

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />

    <meta
      name="viewport"
      content="width=device-width, initial-scale=1.0"
    />

    <meta
      name="description"
      content="Employee Management Portal"
    />

    <title>Employee Management Portal</title>
  </head>

  <body>
    <div id="root"></div>

    <script
      type="module"
      src="/src/main.jsx"
    ></script>
  </body>
</html>
```

File 7: frontend/vite.config.js

JavaScript

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()]
});
```

File 8: frontend/src/main.jsx

JavaScript

```
import React from "react";
import ReactDOM from "react-dom/client";

import App from "./App";
import "./style.css";

ReactDOM.createRoot(
  document.getElementById("root")
).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

File 9: frontend/src/App.jsx

JavaScript

```
import { useEffect, useMemo, useState } from "react";

const API_URL =
  import.meta.env.VITE_API_URL ||
  "http://localhost:5000";

function App() {
  const [employees, setEmployees] = useState([]);
  const [search, setSearch] = useState("");
  const [department, setDepartment] = useState("All");
  const [departments, setDepartments] = useState(["All"]);
  const [loading, setLoading] = useState(true);
```

```

const [error, setError] = useState("");

useEffect(() => {
  loadDepartments();
}, []);

useEffect(() => {
  loadEmployees();
}, [search, department]);

async function loadDepartments() {
  try {
    const response = await fetch(
      `${API_URL}/api/departments`
    );

    if (!response.ok) {
      throw new Error("Unable to load departments");
    }

    const data = await response.json();
    setDepartments(data);
  } catch (err) {
    setError(err.message);
  }
}

async function loadEmployees() {
  try {
    setLoading(true);
    setError("");

    const params = new URLSearchParams();

    if (search.trim()) {
      params.append("search", search.trim());
    }

    if (department !== "All") {
      params.append("department", department);
    }

    const response = await fetch(
      `${API_URL}/api/employees?${params.toString()}`

```

```

);

if (!response.ok) {
  throw new Error("Unable to load employees");
}

const data = await response.json();
setEmployees(data);
} catch (err) {
  setError(
    `${err.message}. Please check the backend connection.`
  );
} finally {
  setLoading(false);
}
}

const totalEmployees = employees.length;

const engineeringEmployees = useMemo(() => {
  return employees.filter(
    (employee) => employee.department === "Engineering"
  ).length;
}, [employees]);

return (
  <div className="app">
    <header className="hero">
      <div>
        <p className="eyebrow">DEVOPS CAPSTONE PROJECT</p>

        <h1>Employee Management Portal</h1>

        <p className="subtitle">
          Explore employee information through a modern
          two-tier web application.
        </p>
      </div>

      <div className="status-card">
        <span className="status-dot"></span>
        Backend API Connected
      </div>
    </header>

```

```
<main className="container">
  <section className="summary-grid">
    <div className="summary-card">
      <span>Total Employees</span>
      <strong>{totalEmployees}</strong>
    </div>

    <div className="summary-card">
      <span>Engineering Team</span>
      <strong>{engineeringEmployees}</strong>
    </div>

    <div className="summary-card">
      <span>Selected Department</span>
      <strong>{department}</strong>
    </div>
  </section>

  <section className="controls">
    <div className="field">
      <label htmlFor="search">Search Employees</label>

      <input
        id="search"
        type="text"
        placeholder="Search by name, role, or email..."
        value={search}
        onChange={(event) =>
          setSearch(event.target.value)
        }
      />
    </div>

    <div className="field">
      <label htmlFor="department">
        Filter by Department
      </label>

      <select
        id="department"
        value={department}
        onChange={(event) =>
          setDepartment(event.target.value)
        }
      />
    </div>
  </section>
</main>
```

```

    }
  >
    {departments.map((item) => (
      <option key={item} value={item}>
        {item}
      </option>
    ))}
  </select>
</div>
</section>

{error && (
  <div className="error-message">
    {error}
  </div>
)}

{loading ? (
  <div className="message">
    Loading employees...
  </div>
) : employees.length === 0 ? (
  <div className="message">
    No employees found.
  </div>
) : (
  <section className="employee-grid">
    {employees.map((employee) => (
      <article
        className="employee-card"
        key={employee.id}
      >
        <div className="avatar">
          {employee.name.charAt(0)}
        </div>

        <div className="employee-content">
          <h2>{employee.name}</h2>

          <p className="role">
            {employee.role}
          </p>

          <div className="employee-details">

```

```

    <p>
      <strong>Email:</strong>{" "}
      {employee.email}
    </p>

    <p>
      <strong>Department:</strong>{" "}
      {employee.department}
    </p>

    <p>
      <strong>Location:</strong>{" "}
      {employee.location}
    </p>

    <p>
      <strong>Experience:</strong>{" "}
      {employee.experience} years
    </p>
  </div>
</div>
</article>
  )))
</section>
})
</main>

<footer className="footer">
  Employee Management Portal · DevOps Capstone
  Project
</footer>
</div>
);
}

```

```
export default App;
```

File 10: frontend/src/style.css

CSS

```

:root {
  font-family:
    Inter,

```

```
ui-sans-serif,  
system-ui,  
-apple-system,  
BlinkMacSystemFont,  
"Segoe UI",  
sans-serif;
```

```
color: #172033;  
background: #f4f7fb;  
font-synthesis: none;  
text-rendering: optimizeLegibility;  
}
```

```
* {  
  box-sizing: border-box;  
}
```

```
body {  
  margin: 0;  
  min-width: 320px;  
}
```

```
button,  
input,  
select {  
  font: inherit;  
}
```

```
.app {  
  min-height: 100vh;  
}
```

```
.hero {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  gap: 24px;  
  padding: 48px 7%;  
  color: white;  
  background:  
    linear-gradient(  
      135deg,  
      #172554,  
      #1d4ed8
```

```
);  
}
```

```
.eyebrow {  
  margin: 0 0 12px;  
  font-size: 0.78rem;  
  font-weight: 700;  
  letter-spacing: 0.15em;  
  opacity: 0.8;  
}
```

```
.hero h1 {  
  margin: 0;  
  font-size: clamp(2rem, 5vw, 4rem);  
  line-height: 1.1;  
}
```

```
.subtitle {  
  max-width: 650px;  
  margin: 18px 0 0;  
  font-size: 1.05rem;  
  line-height: 1.7;  
  opacity: 0.88;  
}
```

```
.status-card {  
  display: flex;  
  align-items: center;  
  gap: 10px;  
  padding: 14px 18px;  
  border: 1px solid rgba(255, 255, 255, 0.3);  
  border-radius: 999px;  
  background: rgba(255, 255, 255, 0.12);  
  white-space: nowrap;  
}
```

```
.status-dot {  
  width: 10px;  
  height: 10px;  
  border-radius: 50%;  
  background: #4ade80;  
}
```

```
.container {
```

```
width: min(1200px, 86%);  
margin: 0 auto;  
padding: 36px 0;  
}
```

```
.summary-grid {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  gap: 20px;  
  margin-bottom: 28px;  
}
```

```
.summary-card {  
  padding: 24px;  
  border: 1px solid #e2e8f0;  
  border-radius: 18px;  
  background: white;  
  box-shadow: 0 8px 24px rgba(15, 23, 42, 0.05);  
}
```

```
.summary-card span {  
  display: block;  
  color: #64748b;  
  font-size: 0.95rem;  
}
```

```
.summary-card strong {  
  display: block;  
  margin-top: 10px;  
  font-size: 2rem;  
  color: #1d4ed8;  
}
```

```
.controls {  
  display: grid;  
  grid-template-columns: 2fr 1fr;  
  gap: 20px;  
  padding: 24px;  
  margin-bottom: 28px;  
  border: 1px solid #e2e8f0;  
  border-radius: 18px;  
  background: white;  
}
```

```
.field {  
  display: flex;  
  flex-direction: column;  
  gap: 8px;  
}
```

```
.field label {  
  font-size: 0.9rem;  
  font-weight: 700;  
  color: #334155;  
}
```

```
.field input,  
.field select {  
  width: 100%;  
  padding: 13px 14px;  
  border: 1px solid #cbd5e1;  
  border-radius: 10px;  
  outline: none;  
  background: white;  
  color: #172033;  
}
```

```
.field input:focus,  
.field select:focus {  
  border-color: #2563eb;  
  box-shadow: 0 0 0 3px rgba(37, 99, 235, 0.12);  
}
```

```
.employee-grid {  
  display: grid;  
  grid-template-columns: repeat(2, 1fr);  
  gap: 20px;  
}
```

```
.employee-card {  
  display: flex;  
  gap: 18px;  
  padding: 24px;  
  border: 1px solid #e2e8f0;  
  border-radius: 18px;  
  background: white;  
  box-shadow: 0 8px 24px rgba(15, 23, 42, 0.05);  
  transition:
```

```
    transform 0.2s ease,  
    box-shadow 0.2s ease;  
}
```

```
.employee-card:hover {  
    transform: translateY(-4px);  
    box-shadow: 0 14px 30px rgba(15, 23, 42, 0.1);  
}
```

```
.avatar {  
    display: grid;  
    place-items: center;  
    flex-shrink: 0;  
    width: 58px;  
    height: 58px;  
    border-radius: 16px;  
    background: #dbeafe;  
    color: #1d4ed8;  
    font-size: 1.5rem;  
    font-weight: 800;  
}
```

```
.employee-content h2 {  
    margin: 0;  
    font-size: 1.25rem;  
}
```

```
.role {  
    margin: 5px 0 18px;  
    color: #2563eb;  
    font-weight: 600;  
}
```

```
.employee-details p {  
    margin: 8px 0;  
    color: #64748b;  
    font-size: 0.92rem;  
    line-height: 1.5;  
    overflow-wrap: anywhere;  
}
```

```
.employee-details strong {  
    color: #334155;  
}
```

```
.message,  
.error-message {  
  padding: 24px;  
  border-radius: 14px;  
  text-align: center;  
  background: white;  
}
```

```
.error-message {  
  margin-bottom: 24px;  
  color: #b91c1c;  
  background: #fee2e2;  
}
```

```
.footer {  
  padding: 28px;  
  text-align: center;  
  color: #64748b;  
  font-size: 0.9rem;  
}
```

```
@media (max-width: 900px) {  
  .hero {  
    flex-direction: column;  
    align-items: flex-start;  
  }  
}
```

```
.summary-grid,  
.employee-grid {  
  grid-template-columns: 1fr;  
}
```

```
.controls {  
  grid-template-columns: 1fr;  
}  
}
```

```
@media (max-width: 560px) {  
  .hero {  
    padding: 36px 7%;  
  }  
}
```

```
.container {
```

```
width: 90%;  
}  
  
.employee-card {  
  flex-direction: column;  
}  
  
.status-card {  
  white-space: normal;  
}  
}
```

PHASE 2 — PUSH CODE TO GITHUB

Students will:

1. Create a GitHub repository named `employee-management-devops`.
2. Initialize Git in the project directory.
3. Add the application files.
4. Commit the source code.
5. Create a `main` branch.
6. Push the project to GitHub.
7. Create a feature branch for future changes.
8. Open a Pull Request and merge it into `main`.

The repository should contain both the `frontend` and `backend` directories.

PHASE 3 — CREATE DOCKERFILES

Students will create:

- A Dockerfile for the backend.
- A Dockerfile for the frontend.
- `.dockerignore` files.
- Docker images for both application components.
- A Docker Compose file to run both services locally.

The frontend should be built using a multi-stage Dockerfile and served through Nginx.

The backend should run as a Node.js container.

Students must verify:

- Both images build successfully.
- Both containers start successfully.
- The frontend can communicate with the backend.
- The application works through Docker Compose.

PHASE 4 — PUSH DOCKER IMAGES TO DOCKER HUB

Students will:

1. Create two Docker Hub repositories.
2. Build the frontend image.
3. Build the backend image.
4. Tag both images correctly.
5. Log in to Docker Hub.
6. Push both images.
7. Verify that the images are available publicly or privately according to the project requirements.

Expected image format:

DOCKERHUB_USERNAME/employee-frontend:1.0

DOCKERHUB_USERNAME/employee-backend:1.0

Students should use versioned image tags rather than relying only on `latest`.

PHASE 5 — BUILD CI/CD PIPELINES

Students will create CI/CD pipelines using:

- Jenkins.
- GitHub Actions.

The pipeline should:

1. Checkout the source code.
2. Install frontend dependencies.
3. Install backend dependencies.
4. Run available tests or validation checks.

5. Build the frontend Docker image.
6. Build the backend Docker image.
7. Log in to Docker Hub using secure credentials.
8. Push both images to Docker Hub.
9. Use a unique image tag such as the build number or Git commit SHA.

Students should configure:

- GitHub webhook for Jenkins.
- Jenkins credentials.
- Docker Hub credentials.
- GitHub Actions secrets.
- Pipeline success and failure notifications.

Secrets must not be written directly inside pipeline files.

PHASE 6 — DEPLOY THE APPLICATION ON KUBERNETES

Students will deploy the application using Kubernetes.

They should create:

- Namespace.
- Frontend Deployment.
- Backend Deployment.
- Frontend Service.
- Backend Service.
- ConfigMap.
- Secret.
- Readiness probe.
- Liveness probe.
- Resource requests and limits.
- Multiple replicas.

The Kubernetes deployment should use the Docker images pushed to Docker Hub.

Students must verify:

- Pods are running.
- Deployments are available.
- Services have correct selectors.
- The frontend can reach the backend.

- Health checks are successful.
- A rolling update works.
- A previous version can be restored using rollback.

PHASE 7 — PROVISION AWS INFRASTRUCTURE USING TERRAFORM

Students will use Terraform to provision the AWS infrastructure required for monitoring.

To keep the infrastructure manageable, they should use the default VPC.

Terraform should create:

- Monitoring EC2 instance.
- Security group.
- SSH key pair.
- Required instance configuration.
- Terraform variables.
- Terraform outputs.

The infrastructure should be designed to monitor the Kubernetes application.

Students must configure suitable access for:

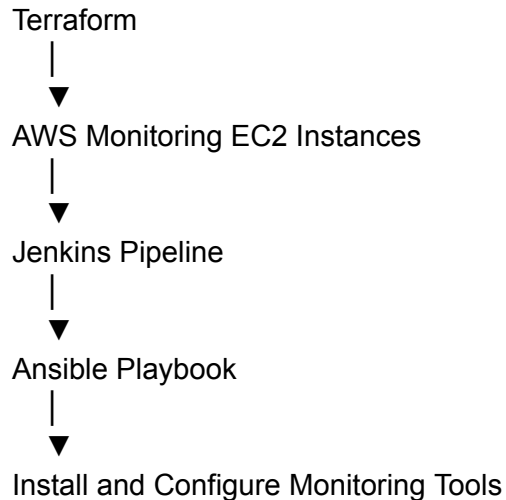
- SSH.
- Prometheus.
- Grafana.
- Node Exporter.
- cAdvisor, if required.
- Application monitoring endpoints.

Students should restrict access to trusted IP addresses wherever possible.

PHASE 8 — USE JENKINS AND ANSIBLE TO INSTALL MONITORING TOOLS

After Terraform creates the AWS machines, students will use Jenkins and Ansible to configure them.

The workflow will be:



Ansible will be responsible for:

- Connecting to the AWS machines through SSH.
- Installing Docker or required Linux packages.
- Installing Prometheus.
- Installing Grafana.
- Installing Node Exporter.
- Installing cAdvisor.
- Creating required directories.
- Copying configuration files.
- Starting and enabling services.
- Checking service status.
- Verifying monitoring endpoints.

Jenkins will execute the Ansible playbooks automatically after the infrastructure is provisioned.

Students should create separate Ansible tasks or roles for:

- Prometheus installation.
- Grafana installation.
- Node Exporter installation.
- cAdvisor deployment.
- Configuration file deployment.
- Service validation.

PHASE 9 — CONFIGURE PROMETHEUS, GRAFANA, AND RELATED TOOLS

Students will configure the monitoring environment to observe the Kubernetes application.

Prometheus

Students will:

- Configure Prometheus scrape intervals.
- Add Prometheus itself as a target.
- Add Node Exporter as a target.
- Add cAdvisor as a target.
- Add Kubernetes-related targets where applicable.
- Verify target health.
- Test PromQL queries.

Node Exporter

Node Exporter will collect Linux host metrics such as:

- CPU usage.
- Memory usage.
- Disk usage.
- Disk space.
- Network traffic.
- System uptime.
- Load average.

cAdvisor

cAdvisor will collect container metrics such as:

- Container CPU usage.
- Container memory usage.
- Container network traffic.
- Container restarts.
- Container resource consumption.

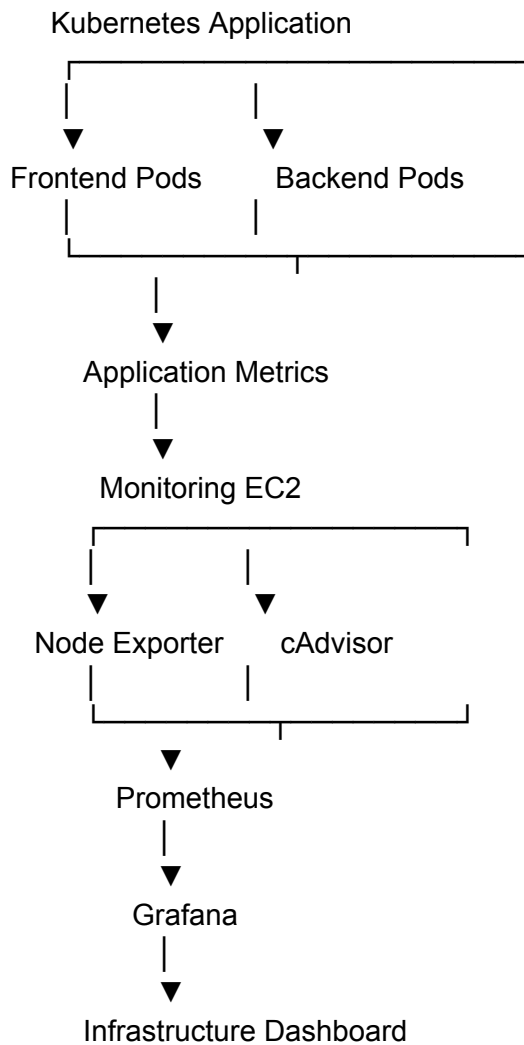
Grafana

Students will:

1. Log in to Grafana.
2. Add Prometheus as a data source.

3. Create an infrastructure dashboard.
4. Create CPU panels.
5. Create memory panels.
6. Create disk panels.
7. Create network panels.
8. Create uptime panels.
9. Create exporter status panels.
10. Create container CPU and memory panels.
11. Configure basic dashboard thresholds.
12. Save and export the dashboard.

Final Monitoring Architecture



Final Project Result

At the end of the project, students will have:

- A working two-tier application.
- A GitHub repository.
- Dockerized frontend and backend.
- Docker Hub images.
- Jenkins CI/CD pipeline.
- GitHub Actions workflow.
- Kubernetes deployment.
- AWS infrastructure created through Terraform.
- AWS servers configured using Ansible.
- Prometheus monitoring.
- Node Exporter host metrics.
- cAdvisor container metrics.
- Grafana infrastructure dashboard.
- A complete industry-style DevOps project portfolio.